

쇼핑몰 백엔드 프로젝트 보고서

1. 프로젝트 개요

프로젝트 소개

본 프로젝트는 Spring Boot 기반의 쇼핑몰 백엔드 API 서버이다. 사용자는 상품을 조회하고 장바구니에 담거나 주문할 수 있으며, 관리자는 상품과 주문, 회원 정보를 관리할 수 있다. 또한 고객 문의를 위한 채팅 기능과 Swagger 기반 API 문서화를 포함한다.

프로젝트 목표

- 쇼핑몰의 기본 업무 흐름인 회원 인증, 상품 조회, 주문, 장바구니 기능을 구현한다.
- JWT 와 Spring Security 를 사용하여 인증과 권한 처리를 적용한다.
- MySQL 과 JPA 를 사용하여 데이터를 영속화한다.
- Docker 와 GitHub Actions 를 이용하여 EC2 배포 자동화 흐름을 구성한다.

주요 기능

- 회원가입 및 로그인
- JWT 기반 인증
- 상품 목록/상세 조회
- 단건 주문 및 장바구니 주문
- 장바구니 담기/조회
- 관리자 상품 등록/수정/삭제
- 관리자 주문/회원 조회
- 고객-관리자 채팅방 및 메시지 관리
- Swagger UI 를 통한 API 확인

2. 요구사항 분석

기능 요구사항

- 사용자는 회원가입과 로그인을 할 수 있다.
- 로그인 성공 시 JWT 를 발급받고, 보호된 API 호출 시 JWT 를 사용한다.
- 누구나 상품 목록과 상품 상세 정보를 조회할 수 있다.

- 로그인한 사용자는 주문을 생성하고 본인의 주문 내역을 조회할 수 있다.
- 로그인한 사용자는 장바구니에 상품을 담고 장바구니 상품을 주문할 수 있다.
- 관리자는 상품을 등록, 수정, 삭제할 수 있다.
- 관리자는 전체 주문과 회원 정보를 조회할 수 있다.
- 사용자는 채팅방을 생성하고 메시지를 조회/전송할 수 있다.

비기능 요구사항

- 인증이 필요한 API 는 JWT 검증을 거쳐야 한다.
- 관리자 기능은 `ROLE\_ADMIN` 권한을 가진 사용자만 접근할 수 있어야 한다.
- DB 비밀번호, JWT Secret 등 민감 정보는 환경변수로 관리한다.
- API 는 Swagger UI 에서 확인 가능해야 한다.
- 애플리케이션은 Docker 컨테이너로 실행 가능해야 한다.

3. 기술 스택

Backend

- Java 21
- Spring Boot
- Spring Web MVC
- Spring Data JPA
- Maven

Database

- MySQL 8.4
- Hibernate

Security

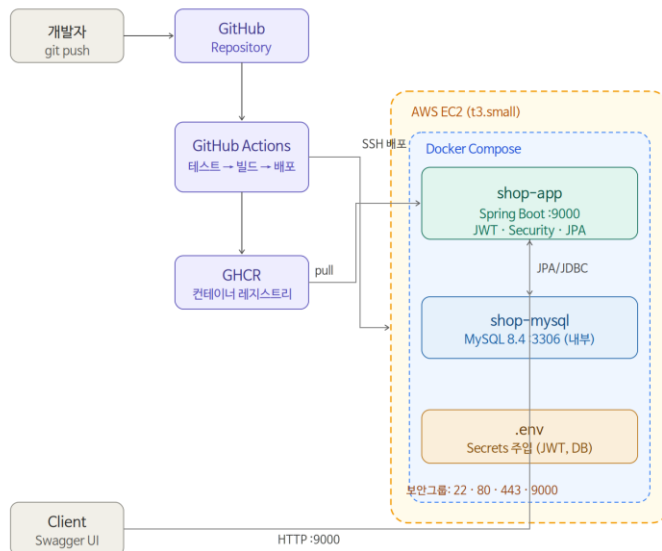
- Spring Security
- JWT
- BCrypt Password Encoder

DevOps

- Docker
- Docker Compose
- GitHub Actions
- GHCR
- AWS EC2

4. 시스템 아키텍처

시스템 구성도



배포 환경에서는 EC2 한 대에서 Spring Boot 컨테이너와 MySQL 컨테이너를 함께 실행한다. Spring Boot 앱은 외부에 9000 번 포트로 공개하고, MySQL 은 외부에 공개하지 않고 Docker 내부 네트워크와 EC2 내부 localhost 에서만 접근하도록 구성한다.

요청 흐름도

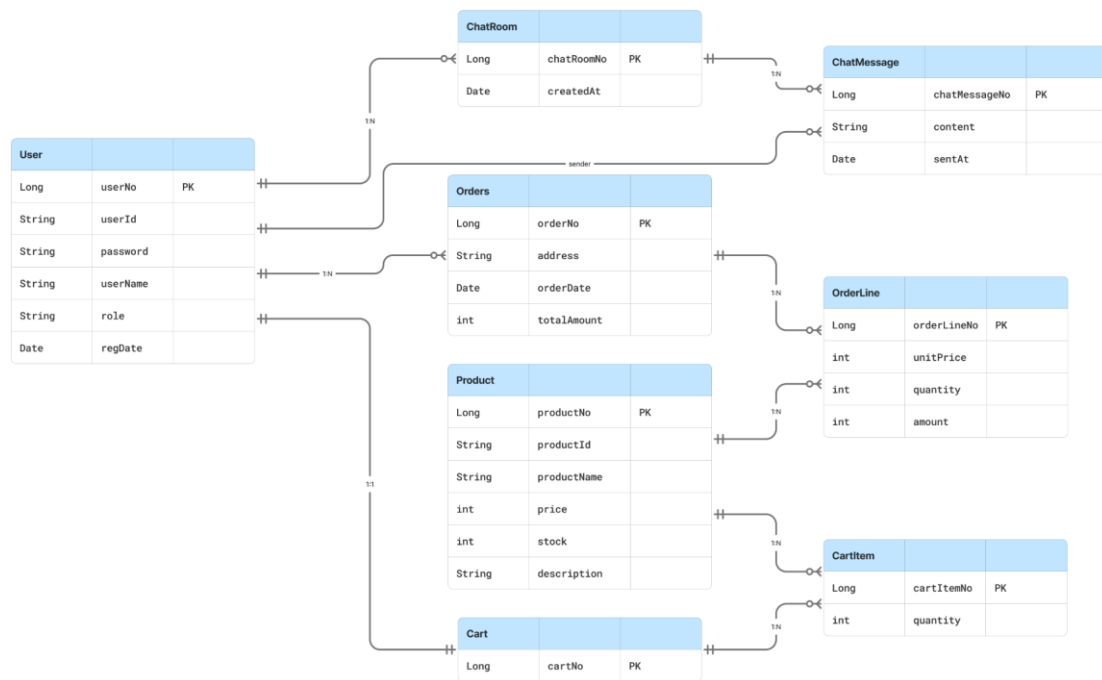
1. 클라이언트가 API 요청
2. Spring Security 필터가 JWT 검증
3. Controller 가 요청 수신
4. Service 에서 비즈니스 로직 처리

5. Repository 를 통해 DB 접근

6. DTO 형태로 응답 반환

5. 데이터베이스 설계

ERD



주요 엔티티는 다음과 같다.

User 1 --- 1 Cart

User 1 --- N Orders

Orders 1 --- N OrderLine

Product 1 --- N OrderLine

Cart 1 --- N CartItem

Product 1 --- N CartItem

User 1 --- N ChatRoom

ChatRoom 1 --- N ChatMessage

User 1 --- N ChatMessage

테이블 정의

테이블명	주요 필드 (Attributes)	설명 (Description)
User	userNo, userId, password, userName, role, regDate	회원 기본 정보 및 권한(Role) 관리
Product	productNo, productId, productName, price, stock, description	판매 상품 정보 및 재고 관리
Orders	orderNo, user, orderDate, address, totalAmount	주문 마스터(기본) 정보
OrderLine	orderLineNo, orders, product, unitPrice, quantity, amount	주문별 상세 상품 및 결제 금액
Cart	cartNo, user	회원별 장바구니 매핑 테이블
CartItem	cartItemNo, cart, product, quantity	장바구니에 담긴 개별 상품 및 수량
ChatRoom	chatRoomNo, user, createdAt	고객 문의 및 상담을 위한 채팅방 정보

6. API 설계

REST API 명세

공통 기능

Method	URI	설명	권한
POST	/register	회원가입	전체
POST	/login	로그인 및 JWT 발급	전체
GET	/api/products	상품 목록 조회	전체

Method	URI	설명	권한
GET	/api/products/{productNo}	상품 상세 조회	전체

고객 기능

Method	URI	설명	권한
POST	/api/orders	단건 주문 생성	로그인
GET	/api/orders/me	내 주문 내역 조회	로그인
POST	/api/cart/item	장바구니 상품 추가	로그인
GET	/api/cart/items	장바구니 조회	로그인
POST	/api/cart/orders	장바구니 상품 전체 주문	로그인
POST	/api/chat/rooms	문의 채팅방 생성	사용자
GET	/api/chat/rooms/{chatRoomNo}/messages	채팅 메시지 내역 조회	로그인
POST	/api/chat/rooms/{chatRoomNo}/messages	채팅 메시지 전송	로그인

관리자 기능

Method	URI	설명	권한
POST	/api/admin/products	신규 상품 등록	관리자
PATCH	/api/admin/products/{productId}	상품 정보 수정 (일부 변경)	관리자

Method	URI	설명	권한
DELETE	/api/admin/products/{productId}	상품 삭제	관리자
GET	/api/admin/orders	전체 사용자 주문 내역 조회	관리자
GET	/api/admin/users	전체 회원 정보 조회	관리자
GET	/api/admin/chat/rooms	활성화된 전체 채팅방 목록 조회	관리자

API 규칙

- 요청/응답은 JSON 형식을 사용한다.
- 공통 응답 형식으로 `ApiResponse`를 사용한다.
- 인증이 필요한 요청은 `Authorization: Bearer {token}` 헤더를 사용한다.
- 예외 발생 시 `ErrorResponse`를 통해 에러 정보를 반환한다.

7. 프로젝트 구조

패키지 구조

```

web.mvc
├─ advice
├─ config
├─ controller
├─ domain
├─ dto
│   └─ request
│   └─ response
├─ exception
├─ jwt
├─ repository
├─ security
└─ service

```

계층 구조

- Controller: HTTP 요청을 받고 Service 를 호출한다.
- Service: 주문 생성, 장바구니 처리, 상품 관리 등 비즈니스 로직을 담당한다.
- Repository: JPA 를 통해 DB 에 접근한다.
- Domain: JPA Entity 를 정의한다.
- DTO: 요청/응답 데이터를 분리한다.

8. 회원 인증 및 인가

회원가입

/register API 를 통해 회원을 생성한다. 비밀번호는 BCrypt 로 암호화하여 저장한다.

로그인

/login 요청 시 Spring Security 의 인증 과정을 거치고, 인증 성공 시 JWT 를 발급한다.

JWT 인증

발급된 JWT 는 이후 API 요청에서 Authorization 헤더에 포함된다. JWTFilter 가 토큰을 검증하고 인증 정보를 SecurityContext 에 저장한다.

권한 처리

Spring Security 설정에서 API 별 접근 권한을 제어한다.

- 상품 조회, Swagger: 전체 허용
- 주문/장바구니: 로그인 사용자
- 관리자 상품/주문/회원 API: `ROLE\_ADMIN`
- 채팅방 생성: `ROLE\_USER`

9. 핵심 비즈니스 기능 구현

CRUD 기능

- 상품 등록, 수정, 삭제

- 상품 조회
- 회원 등록
- 주문 생성
- 장바구니 추가 및 조회
- 채팅방/메시지 생성 및 조회

검색 기능

현재 별도의 검색 API 는 핵심 범위에 포함하지 않았다. 상품 목록 조회와 상세 조회를 중심으로 구현하였다.

10. JPA 활용

엔티티 설계

User, Product, Orders, OrderLine, Cart, CartItem, ChatRoom, ChatMessage 엔티티를 중심으로 설계하였다.

연관관계 매핑

- `User`와 `Orders`: 다대일
- `Orders`와 `OrderLine`: 일대다
- `Product`와 `OrderLine`: 다대일
- `User`와 `Cart`: 일대일
- `Cart`와 `CartItem`: 일대다
- `ChatRoom`과 `ChatMessage`: 일대다

Lazy Loading

회원, 주문, 장바구니, 채팅방 등 주요 연관관계에 FetchType.LAZY 를 적용하여 불필요한 즉시 로딩을 줄였다.

Query Method

Repository 에서 JPA Query Method 와 JPQL 을 사용한다.

- `findByUserId`
- `existsByUserId`
- `findByProductId`
- `findCartsByUserId`
- `findAllByUserIdWithOrderLines`

11. 테스트

단위 테스트

주요 비즈니스 로직은 서비스 계층을 중심으로 테스트할 수 있다. 예를 들어 상품 조회, 주문 생성, 장바구니 추가, 재고 차감 로직이 테스트 대상이다.

API 테스트

GitHub Actions 에서 MySQL service 컨테이너를 띄운 뒤 Maven 테스트를 실행한다.

Test Job

→ MySQL 컨테이너 실행

→ Java 21 설정

→ `./mvnw -q test`

Swagger UI 를 통해 수동 API 테스트도 수행하였다.

12. API 문서화

Swagger(OpenAPI)

springdoc-openapi 를 사용하여 Swagger UI 를 제공한다.

접속 경로: `/swagger`

배포 환경에서는 다음 형식으로 접근한다.

`http://{EC2-public-ip}:9000/swagger`

13. Docker 적용

Dockerfile 작성

멀티 스테이지 빌드를 사용한다.

1. Maven 이미지에서 jar 빌드
2. Java 21 runtime 이미지로 app.jar 복사
3. java -jar /app/app.jar 실행

컨테이너 실행

Docker Compose 로 두 개의 컨테이너를 실행한다.

- `shop-app`: Spring Boot 앱
- `shop-mysql`: MySQL DB

Spring Boot 는 mysql:3306 으로 DB 에 접근한다. MySQL 은 외부에 직접 공개하지 않고 내부 통신 중심으로 사용한다.

14. 배포

AWS EC2 배포

최종 배포는 수동으로 생성한 EC2 인스턴스에 GitHub Actions 가 SSH 로 접속하여 수행한다.

GitHub Actions 배포 흐름:

1. 테스트 실행
2. Docker 이미지 빌드
3. GHCR 에 이미지 push
4. EC2 SSH 접속
5. Docker / Docker Compose 설치 확인
6. /opt/shop/.env 생성
7. /opt/shop/docker-compose.yml 생성

8. docker compose pull
9. docker compose up -d

10. EC2 내부에서 /swagger 응답 확인

필수 GitHub Secrets:

EC2_HOST
EC2_USER
EC2_SSH_KEY
ADMIN_USER_ID
ADMIN_PASSWORD

서비스 실행 확인

배포 후 Swagger 페이지로 서비스 실행을 확인한다.

<http://15.164.210.175:9000/swagger>

15. 트러블슈팅

1. DTO 사용 기준 혼동

초기에는 DTO에 대한 이해가 부족하여 Entity, 요청 DTO, 응답 DTO를 어느 계층에서 사용해야 하는지 혼동이 있었다. 특히 상품 조회 기능에서 Product, ProductReq, ProductRes의 역할이 섞이면서 Service 반환 타입과 Controller 응답 타입을 정리할 필요가 있었다.

해결:

- 요청 데이터는 Request DTO로 받도록 정리하였다.
- 응답 데이터는 Response DTO로 반환하도록 정리하였다.
- Entity를 API 응답에 직접 노출하지 않고 `ProductRes` 같은 응답 DTO를 사용하도록 방향을 잡았다.
- 공통 응답은 `ApiResponse<T>`로 감싸 일관성을 유지하였다.

2. Docker Compose 설치 실패

Amazon Linux 2023 기반 EC2에서 user_data 실행 중 Docker Compose 설치가 실패했다.
No match for argument: docker-compose-plugin

Error: Unable to find a match: docker-compose-plugin

원인은 `dnf install docker-compose-plugin` 으로 설치 가능한 패키지가 기본 저장소에 없었기 때문이다.

해결:

- Docker Compose v2 바이너리를 직접 다운로드하여 설치하는 방식으로 변경하였다.
- 이후 수동 EC2 배포로 전환하면서 GitHub Actions 가 Docker 와 Docker Compose 설치 여부를 직접 확인하도록 구성하였다.

3. EC2 접속 및 보안그룹 문제

초기에는 Terraform 으로 인프라를 구성하려고 했지만 SSH 와 애플리케이션 포트 접근이 되지 않아 보안그룹, Route Table, NACL, Public IP 를 점검하였다. 이후 문제 해결 시간이 길어지면서 최종적으로 수동 EC2 방식으로 전환하였다.

해결:

- EC2 를 수동으로 생성하였다.
- 보안그룹에서 필요한 포트를 명시적으로 열었다.
- GitHub Secrets 에는 ``EC2_HOST``, ``EC2_USER``, ``EC2_SSH_KEY``를 등록하였다.

열어둔 포트:

22 SSH

80 HTTP

443 HTTPS 또는 SSH 대체 접속

9000 Spring Boot

4. Terraform Provisioning 의존성 제거

기존 배포 흐름은 Terraform `user_data` 가 EC2 내부에 `/opt/shop/.env` 와 `/opt/shop/docker-compose.yml` 을 미리 만들어준다는 전제에 의존하고 있었다. 수동 EC2 로 전환하면서 이 의존성을 제거해야 했다.

해결:

- ``deploy.yml``에서 Docker 설치 여부를 확인하도록 수정하였다.
- Docker Compose 가 없으면 설치하도록 구성하였다.

- `/opt/shop/.env`와 `/opt/shop/docker-compose.yml`을 GitHub Actions 배포 단계에서 직접 생성하도록 변경하였다.
- 결과적으로 수동 EC2 에서도 필요한 Secrets 만 등록하면 배포할 수 있게 되었다.

5. EC2 자원 부족 문제

EC2 한 대에서 MySQL 과 Spring Boot 를 함께 실행하면서 CPU 와 메모리 사용량이 높아졌다. t3.micro 는 자원이 작기 때문에 컨테이너 실행 중 서버가 불안정해질 수 있었다.

대응:

- MobaXterm 에서 리소스 사용량을 확인하였다.
- 필요 시 swap 추가를 고려하였다.
- 최종적으로 더 안정적인 실행을 위해 `t3.small` 사용을 선택하였다.

6. 외부 브라우저 접속 실패

GitHub Actions 에서는 배포가 완료되었지만 브라우저에서 Swagger 가 열리지 않는 문제가 있었다. 이때는 앱 자체 문제와 외부 네트워크 접근 문제를 분리해서 확인해야 했다.

확인 기준:

- EC2 내부에서 `http://127.0.0.1:9000/swagger`가 열리면 앱은 정상 실행 중이다.
- 외부에서 `http://<EC2\_PUBLIC\_IP>:9000/swagger`가 열리지 않으면 보안그룹, Public IP, NACL, Route Table 을 확인해야 한다.

해결:

- 올바른 Public IP 와 9000 포트를 사용하였다.
- 보안그룹에 9000 포트 인바운드를 열어 외부 접속을 가능하게 했다.

7. Admin 계정 로그인 실패

서비스 접속은 되었지만 admin 계정으로 로그인할 수 없었다. 원인은 admin 계정이 코드에 고정되어 있지 않고, 환경변수로 주입될 때만 자동 생성되는 구조였기 때문이다.

Admin 계정 생성에 필요한 값:

ADMIN_USER_ID

ADMIN_PASSWORD

해결:

- GitHub Secrets 에 `ADMIN\_USER\_ID`, `ADMIN\_PASSWORD`를 추가하였다.
- 재배포 후 `AdminAccountInitializer`가 admin 계정을 생성하였다.
- 같은 ID의 사용자가 이미 존재하면 새로 생성하지 않으므로 기존 DB 상태도 함께 확인하였다.

16. 프로젝트 회고

배운점

- 수업을 듣고 어느정도 MVC 구조에 대해서 이해했다고 생각했는데 막상 백지 위에 올라오니 코드 하나도 생각이 잘 나지 않았다. 코드를 읽을 때는 쉬워 보이던 것들이 코드를 작성하려고 보니 어렵게만 느껴졌다.
개인 프로젝트를 통해 백엔드 흐름에 대해서 공부하니까 가볍게 지나쳤던 구간들을 다시 자세하게 볼 수 있었다.
- DTO 관련한 부분이 그러했다. DTO가 막연히 입력값과 출력값을 엔티티 객체로 보내지 않고 내가 원하는 정보만 가지고 주고 받을 수 있게 한다고 생각했는데, 실제 작업에서는 DTO에 필요한 것이 무엇인지 왜 request와 response로 나눠야 하는지를 확실하게 이해할 수 있었다. 또한, dto를 어디서 작업해서 req -> domain -> res로 변환해야 하는지도 이해하게 되었다.
- 그래도 아직 복잡한 쿼리문을 생성하는 데 있어서는 어려움을 느끼고 있어서 앞으로 더 공부해야 할 부분인 거 같다.

개선할 점

- terraform을 사용해서 인프라를 관리하려고 했는데 코드를 직접 작성하는 게 아니었음에도 여러가지 에러를 만나서 결국 간단한 프로젝트이기 때문에 수동으로 인프라를 생성하게 되었다.
- 관리자 기능과 일반 고객 기능을 좀 더 확실하게 나눌 필요가 있다. 지금은 관리자도 상품 관련 기능들을 사용할 수 있게 했지만 관리자는 관리자로서의 기능만 담당할 수 있게 경계를 확실히 해야겠다.

향후 계획

1. 구매 상품 리뷰 기능

구매 이력이 있는 사용자만 해당 상품에 리뷰를 작성할 수 있도록 기능을 추가할 계획이다. 리뷰에는 별점과 내용을 포함하고, 상품 상세 조회 시 리뷰 목록과 평균 평점을 함께 제공하여 사용자의 구매 의사결정을 돕는다. 또한 사용자가 본인이 작성한 리뷰를 수정하거나 삭제할 수 있도록 구현할 예정이다.

2. 찜하기 기능

사용자가 관심 있는 상품을 저장할 수 있도록 찜하기 기능을 추가할 계획이다. 상품 목록이나 상세 페이지에서 찜 등록/해제를 할 수 있고, 마이페이지에서 찜한 상품 목록을 조회할 수 있도록 한다. 이를 통해 사용자가 관심 상품을 다시 찾기 쉽게 하여 쇼핑 편의성을 높일 수 있다.

3. 페이징 처리 적용

현재 상품, 주문, 회원, 채팅 메시지 목록은 데이터가 많아질 경우 응답 속도가 느려질 수 있다. 이를 개선하기 위해 Pageable 기반의 페이징 처리를 적용할 계획이다. 상품 목록, 관리자 주문 조회, 회원 목록 조회, 채팅 메시지 조회 등에 페이징을 적용하여 성능과 사용자 경험을 개선한다.

4. 배포 안정화

현재는 단일 EC2 인스턴스에서 Docker Compose 로 Spring Boot 와 MySQL 을 함께 실행하는 구조이다. 향후에는 Nginx reverse proxy 와 HTTPS 인증서를 적용하여 보안성을 높이고, Docker 로그 관리와 DB 백업 자동화를 추가할 계획이다. 또한 배포 실패 시 원인을 쉽게 파악할 수 있도록 GitHub Actions 로그와 헬스체크를 개선하고, 무중단 배포 방식도 검토한다.